

INTERNSHIP PROJECT REPORT

InternSpark AICTE-Listed Java Internship Program

TASK 3

Unit Testing & Logging

using Spring Boot · JUnit 5 · Mockito · SLF4J / Logback

Submitted By Swapnil Soni

College VIT Bhopal University

Branch B.Tech Computer Science Engineering

Program InternSpark AICTE-Listed Internship
Program

Declaration

I, Swapnil Soni, a student of B.Tech Computer Science Engineering at VIT Bhopal University, hereby declare that this project report titled "Unit Testing & Logging" submitted as part of Task 3 of the InternSpark AICTE-listed Java Internship Program is an original and authentic record of work undertaken by me.

The work reported in this document has been carried out under the guidelines of the InternSpark program. All references to external sources, tools, frameworks, and libraries have been duly acknowledged. No part of this report has been submitted for any other examination or award.

Swapnil Soni

B.Tech CSE, VIT Bhopal University
InternSpark — Task 3

Table of Contents

1. Introduction	3
2. Objectives	4
3. Technology Stack	5
4. Project Architecture	6
5. Project Structure	7
6. Database Configuration	8
7. API Endpoints	9
8. Unit Testing Explanation	10
9. Logging Explanation	12
10. Screenshots Section	13
11. Challenges Faced	14
12. Learning Outcomes	15
13. Conclusion	16
14. References	17

1. Introduction

This report documents the work completed as part of Task 3 of the InternSpark AICTE-listed Java Internship Program. Building upon the database integration and schema migration foundations established in Task 2, this task focuses on two critical pillars of professional software engineering: Unit Testing and Application Logging.

Unit testing is a software testing methodology where individual units or components of an application are tested in isolation to verify that each one behaves as expected. In enterprise Java development, JUnit 5 is the de facto standard for writing and executing unit tests, while Mockito is the most widely adopted mocking framework that enables developers to simulate dependencies and test components independently of their collaborators.

Logging, on the other hand, is the practice of systematically recording runtime information — informational messages, warnings, errors, and debug traces — to facilitate troubleshooting, performance monitoring, and audit trails. SLF4J (Simple Logging Facade for Java) provides a unified logging abstraction layer, while Logback serves as the underlying logging implementation, offering powerful configuration and output capabilities.

This project builds a complete Spring Boot REST API with MySQL database integration, implements a comprehensive unit testing suite using JUnit 5 and Mockito to cover the Service and Controller layers, and integrates SLF4J-based structured logging throughout the application. The API exposes two endpoints — GET /students and POST /students — which are tested using Postman. The project follows a clean layered architecture, ensuring separation of concerns and maintainability.

2. Objectives

The primary objectives of this task are to:

- Design and build a complete Spring Boot REST API with MySQL integration, following production-ready layered architecture patterns.
- Implement unit testing using JUnit 5 and Mockito to achieve meaningful coverage of the Service and Controller layers, ensuring that business logic is validated independently of the database and external systems.
- Write mocked unit tests that verify:
 - Correct retrieval of all student records (GET /students).
 - Correct persistence of a new student record (POST /students).
 - Appropriate method invocation and interaction contracts between layers.
- Integrate structured logging using SLF4J and Logback across all application layers, recording lifecycle events, request details, and error conditions at appropriate log levels.
- Expose and validate two REST API endpoints using Postman — GET /students for retrieving all records, and POST /students for creating new student entries.
- Follow clean project structure and maintain consistent coding standards aligned with industry best practices for Spring Boot application development.
- Apply version control discipline using Git, commit with descriptive messages, and publish the completed project to a remote GitHub repository.

3. Technology Stack

The following technologies and tools were used throughout the development, testing, and validation of this project:

Technology / Tool	Version / Category	Purpose in This Project
Java	17 LTS	Core programming language for all backend business logic, entity definitions, and API implementation.
Spring Boot	3.x	Primary framework for rapidly bootstrapping and running a production-grade REST API with embedded Tomcat.
Spring Data JPA	Included with Boot	ORM abstraction layer that maps the Student entity to the MySQL students table via JpaRepository.
MySQL	8.x	Relational database for persistent storage of student records in the task3_db schema.
JUnit 5	Jupiter 5.x	Standard Java unit testing framework used to define test cases, assertions, and test lifecycle management.
Mockito	5.x	Mocking framework used to isolate Service and Controller layers from their dependencies during unit tests.
SLF4J	2.x	Logging facade providing a uniform API for application logging, decoupled from the underlying implementation.
Logback	1.4.x	Concrete SLF4J logging backend bundled with Spring Boot; handles log level routing, formatting, and output targets.
Lombok	1.18.x	Annotation processor that auto-generates boilerplate Java code such as getters, setters, and constructors.
Maven	3.x	Build automation and dependency management tool; manages the project lifecycle and classpath.
Postman	Latest	REST API testing client used to send HTTP requests and inspect responses for both GET and POST endpoints.
Git & GitHub	Latest	Version control system and remote repository platform for source code management and submission.
IntelliJ IDEA	Community / Ultimate	Primary integrated development environment used for coding, running, debugging, and testing the application.

4. Project Architecture

This project follows the standard Spring Boot layered (N-tier) architecture that cleanly separates concerns across distinct horizontal layers. Each layer has a well-defined responsibility and communicates only with its immediate neighbours, which enables independent unit testing of each layer using mocked collaborators.

Layer	Class / Component	Responsibility
Presentation (Controller)	StudentController	Handles incoming HTTP requests, maps URL paths to handler methods, delegates to the Service layer, and serialises Java objects to JSON responses.
Business Logic (Service)	StudentService	Encapsulates all business rules, orchestrates interactions between the Controller and Repository, and is the primary target for unit testing with Mockito.
Data Access (Repository)	StudentRepository	Extends JpaRepository<Student, Long> to inherit CRUD operations. Mocked in unit tests to isolate the Service layer from the actual database.
Domain (Entity)	Student	A JPA-annotated POJO that maps to the students table in MySQL. Fields include id, name, email, and course. Lombok annotations remove boilerplate code.
Database	MySQL (task3_db)	Relational persistence layer. The students table stores all student records inserted via the POST endpoint and retrieved via the GET endpoint.
Testing	StudentServiceTest, StudentControllerTest	JUnit 5 test classes with Mockito mocks that exercise Service and Controller methods in isolation from the database and HTTP stack.
Logging	SLF4J + Logback	Cross-cutting concern applied at all layers via @Slf4j annotations. Records application events at INFO, DEBUG, WARN, and ERROR levels.

5. Project Structure

The project adheres to the standard Maven directory layout and Spring Boot package conventions. The source tree is organised as follows:

```
unit-testing-logging-demo/
├── src/
│   ├── main/
│   │   ├── java/com/example/demo/
│   │   │   ├── controller/
│   │   │   │   └── StudentController.java
│   │   │   ├── entity/
│   │   │   │   └── Student.java
│   │   │   ├── repository/
│   │   │   │   └── StudentRepository.java
│   │   │   ├── service/
│   │   │   │   └── StudentService.java
│   │   │   └── DemoApplication.java
│   │   └── resources/
│   │       ├── application.properties
│   │       └── logback-spring.xml
│   └── test/
│       ├── java/com/example/demo/
│       │   ├── StudentServiceTest.java
│       │   └── StudentControllerTest.java
└── pom.xml
```

5.1 Key File Descriptions

File	Description
StudentController.java	Exposes GET /students and POST /students HTTP endpoints using @RestController and @RequestMapping.
Student.java	JPA entity class annotated with @Entity and @Table(name="students"). Lombok @Data, @NoArgsConstructor, @AllArgsConstructor reduce boilerplate.
StudentRepository.java	Interface extending JpaRepository<Student, Long>; provides inherited save(), findAll(), and other CRUD operations automatically.

StudentService.java	Service class injected with StudentRepository. Contains getAllStudents() and saveStudent(Student s) methods, both instrumented with SLF4J logging statements.
DemoApplication.java	Entry point; contains the main() method that bootstraps the Spring application context.
application.properties	Configures MySQL JDBC URL, credentials, JPA settings, and Hibernate DDL auto-creation for the task3_db database.
logback-spring.xml	Logback configuration file that sets log levels, defines the console and rolling-file appenders, and formats log output with timestamps and thread names.
StudentServiceTest.java	JUnit 5 test class using @ExtendWith(MockitoExtension.class) with a mocked StudentRepository to unit-test the Service layer.
StudentControllerTest.java	JUnit 5 test class using @WebMvcTest with MockMvc and a mocked StudentService to unit-test the Controller layer without starting a full server.

6. Database Configuration

The application connects to a locally hosted MySQL instance using JDBC. Spring Boot auto-configuration handles connection pooling via HikariCP. The schema is created automatically by Hibernate on first startup using `spring.jpa.hibernate.ddl-auto=create` (or `update` for subsequent runs), eliminating the need for manual table creation in this task.

6.1 application.properties

```
# — MySQL Datasource —————
spring.datasource.url=jdbc:mysql://localhost:3306/task3_db
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# — JPA / Hibernate —————
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

# — Logging —————
logging.level.com.example.demo=DEBUG
logging.level.org.springframework.web=INFO
logging.file.name=logs/application.log
```

6.2 Database Details

Property	Value
Database Name	task3_db
Table Name	students
Primary Key	id (BIGINT, AUTO_INCREMENT)
Columns	id, name (VARCHAR 100), email (VARCHAR 150, UNIQUE), course (VARCHAR 100)
DDL Strategy	spring.jpa.hibernate.ddl-auto=update (Hibernate manages schema)
Connection Pool	HikariCP (Spring Boot default)
JDBC Driver	com.mysql.cj.jdbc.Driver

7. API Endpoints

The application exposes two REST API endpoints via the StudentController. Both endpoints are mapped under the /students base path and communicate over HTTP on the default Spring Boot port 8080.

Method	Endpoint	Description	Request Body
GET	/students	Retrieves a complete list of all student records persisted in the students table.	None
POST	/students	Creates and persists a new student record. Returns the saved entity with the auto-generated id.	JSON body: { name, email, course }

7.1 Sample Request & Response

POST /students — Request Body

```
POST http://localhost:8080/students
Content-Type: application/json

{
  "name": "Swapnil",
  "email": "swapnil@gmail.com",
  "course": "CSE"
}
```

POST /students — Response (201 Created)

```
{
  "id": 1,
  "name": "Swapnil",
  "email": "swapnil@gmail.com",
  "course": "CSE"
}
```

GET /students — Response (200 OK)

```
[
  {
    "id": 1,
```

```
"name": "Swapnil",  
"email": "swapnil@gmail.com",  
"course": "CSE"  
}  
]
```

8. Unit Testing Explanation

Unit testing is a fundamental software quality assurance practice in which individual components ("units") of the source code are tested in isolation to confirm they behave correctly. In the context of a Spring Boot application, units are typically individual method calls within Service or Controller classes. The goal is to verify logic correctness without relying on a running database, HTTP server, or any other external system.

8.1 Frameworks Used

- JUnit 5 (Jupiter) — Provides the `@Test` annotation, assertion methods (`assertEquals`, `assertNotNull`, `verify` interactions), and the test lifecycle (`@BeforeEach`, `@AfterEach`). JUnit 5 is the modern successor to JUnit 4 and is natively supported by Spring Boot Test starter.
- Mockito — A mocking framework that allows creation of mock objects for any interface or class. When a Service is tested, its `StudentRepository` dependency is replaced with a Mockito mock that returns pre-programmed responses without touching the real database.
- `@WebMvcTest` — A Spring Test slice annotation that loads only the web layer (Controller beans) without loading the full application context, enabling lightweight Controller testing using `MockMvc`.

8.2 StudentServiceTest

The `StudentServiceTest` class tests the `StudentService` layer in complete isolation from the database by mocking `StudentRepository` using Mockito.

```
@ExtendWith(MockitoExtension.class)

class StudentServiceTest {

    @Mock
    private StudentRepository studentRepository;

    @InjectMocks
    private StudentService studentService;

    @Test
    void testGetAllStudents() {

        Student s = new Student(1L, "Swapnil", "swapnil@gmail.com", "CSE");

        when(studentRepository.findAll()).thenReturn(List.of(s));

        List<Student> result = studentService.getAllStudents();
    }
}
```

```
        assertEquals(1, result.size());
        assertEquals("Swapnil", result.get(0).getName());
        verify(studentRepository, times(1)).findAll();
    }

    @Test
    void testSaveStudent() {
        Student s = new Student(null, "Swapnil", "swapnil@gmail.com", "CSE");
        Student saved = new Student(1L, "Swapnil", "swapnil@gmail.com", "CSE");
        when(studentRepository.save(s)).thenReturn(saved);

        Student result = studentService.saveStudent(s);

        assertNotNull(result.getId());
        assertEquals(1L, result.getId());
        verify(studentRepository, times(1)).save(s);
    }
}
```

8.3 StudentControllerTest

The StudentControllerTest class uses `@WebMvcTest` to load only the Controller layer and `MockMvc` to simulate HTTP requests without starting a full web server. The StudentService is mocked using `@MockBean`.

```
@WebMvcTest(StudentController.class)
class StudentControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private StudentService studentService;

    @Test
    void testGetAllStudents() throws Exception {
```

```

Student s = new Student(1L, "Swapnil", "swapnil@gmail.com", "CSE");
when(studentService.getAllStudents()).thenReturn(List.of(s));

mockMvc.perform(get("/students"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$[0].name").value("Swapnil"));
}

@Test
void testSaveStudent() throws Exception {
    Student s = new Student(1L, "Swapnil", "swapnil@gmail.com", "CSE");
    when(studentService.saveStudent(any())).thenReturn(s);

    mockMvc.perform(post("/students")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"name\":\"Swapnil\",\"email\":\"swapnil@gmail.com\",\"course\":\"CSE\"}"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value(1));
}
}

```

8.4 Test Coverage Summary

Test Class	Method Under Test	Assertion Verified	Mock Used
StudentServiceTest	getAllStudents()	List size, student name, findAll() called once	StudentRepository
StudentServiceTest	saveStudent(student)	Non-null id, correct id value, save() called once	StudentRepository
StudentControllerTest	GET /students	HTTP 200 OK, JSON name field matches	StudentService
StudentControllerTest	POST /students	HTTP 200 OK, JSON id field matches	StudentService

9. Logging Explanation

Logging is a cross-cutting concern that provides real-time and historical visibility into application behaviour, enabling developers and operations teams to diagnose issues, trace request flows, and monitor system health. In this project, SLF4J is used as the logging facade and Logback as the concrete implementation — a combination that is the Spring Boot default and industry standard.

9.1 Log Levels Used

Level	When Used	Example Message
DEBUG	Fine-grained diagnostic information useful during development.	"Fetching all students from repository"
INFO	High-level lifecycle events: application startup, request receipt, successful operations.	"GET /students called — returning N records"
WARN	Non-critical issues that warrant attention but do not halt execution.	"Student email already exists, duplicate attempted"
ERROR	Exceptions or failure conditions that require immediate investigation.	"Failed to save student: database connection lost"

9.2 StudentService — Logging Implementation

```
@Service
@Slf4j
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public List<Student> getAllStudents() {
        log.info("getAllStudents() called — fetching records from database");
        List<Student> students = studentRepository.findAll();
        log.debug("Retrieved {} student record(s)", students.size());
        return students;
    }
}
```



```
public Student saveStudent(Student student) {  
    log.info("saveStudent() called for: {}", student.getEmail());  
    Student saved = studentRepository.save(student);  
    log.info("Student saved successfully with id: {}", saved.getId());  
    return saved;  
}  
}
```

9.3 Logback Configuration (logback-spring.xml)

A custom logback-spring.xml file is placed under src/main/resources to override Spring Boot's default logging configuration and enable a rolling-file appender alongside the console appender:

```
<configuration>  
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">  
        <encoder>  
            <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} -  
            %msg%n</pattern>  
        </encoder>  
    </appender>  
  
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">  
        <file>logs/application.log</file>  
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">  
            <fileNamePattern>logs/application.%d{yyyy-MM-dd}.log</fileNamePattern>  
            <maxHistory>7</maxHistory>  
        </rollingPolicy>  
        <encoder>  
            <pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n</pattern>  
        </encoder>  
    </appender>  
  
    <root level="INFO">  
        <appender-ref ref="CONSOLE" />  
        <appender-ref ref="FILE" />  
    </root>
```

```
<logger name="com.example.demo" level="DEBUG" additivity="false">
  <appender-ref ref="CONSOLE" />
  <appender-ref ref="FILE" />
</logger>
</configuration>
```

10. Screenshots Section

The following figures document each major phase of the project — from the IDE project structure to Postman API testing and unit test results. Replace each placeholder with the corresponding actual screenshot before submitting.

Figure 1 — Project Structure in IntelliJ IDEA

[Screenshot Placeholder]

Maven project tree in IntelliJ IDEA showing controller, entity, repository, service, test, and resources packages

Figure 1: Maven project tree in IntelliJ IDEA showing controller, entity, repository, service, test, and resources packages

Figure 2 — application.properties Configuration

[Screenshot Placeholder]

application.properties file showing MySQL datasource URL, JPA settings, and logging configuration

Figure 2: application.properties file showing MySQL datasource URL, JPA settings, and logging configuration

Figure 3 — Student Entity & Repository Classes

[Screenshot Placeholder]

StudentEntity.java and StudentRepository.java open in the editor

Figure 3: StudentEntity.java and StudentRepository.java open in the editor

Figure 4 — StudentService with SLF4J Logging

[Screenshot Placeholder]

StudentService.java showing @Slf4j annotation and log.info / log.debug statements

Figure 4: StudentService.java showing @Slf4j annotation and log.info / log.debug statements

Figure 5 — Unit Test Classes (StudentServiceTest & StudentControllerTest)

[Screenshot Placeholder]

JUnit 5 test files with Mockito annotations visible in the editor

Figure 5: JUnit 5 test files with Mockito annotations visible in the editor

Figure 6 — Unit Tests Passing in IntelliJ IDEA

[Screenshot Placeholder]

IntelliJ IDEA test runner showing all 4 unit tests passing with green checkmarks

Figure 6: IntelliJ IDEA test runner showing all 4 unit tests passing with green checkmarks

Figure 7 — Application Startup Logs

[Screenshot Placeholder]

IntelliJ console showing Spring Boot startup logs with INFO-level messages from SLF4J

Figure 7: IntelliJ console showing Spring Boot startup logs with INFO-level messages from SLF4J

Figure 8 — GET /students Postman Request & Response

[Screenshot Placeholder]

Postman showing GET http://localhost:8080/students with 200 OK and JSON array response

Figure 8: Postman showing GET http://localhost:8080/students with 200 OK and JSON array response

Figure 9 — POST /students Postman Request & Response

[Screenshot Placeholder]

Postman showing POST http://localhost:8080/students with JSON body and 200 OK response containing the saved student with id

Figure 9: Postman showing POST http://localhost:8080/students with JSON body and 200 OK response containing the saved student with id

Figure 10 — students Table in MySQL Workbench

[Screenshot Placeholder]

MySQL Workbench query result showing all rows in the students table of task3_db

Figure 10: MySQL Workbench query result showing all rows in the students table of task3_db

11. Challenges Faced

The following challenges were encountered during the development and testing phases of this project, along with the approaches taken to resolve them:

11.1 MockBean Injection in WebMvcTest

When writing the `StudentControllerTest` using `@WebMvcTest`, the `StudentService` bean was not automatically available in the stripped-down application context loaded by the test slice. Attempting to autowire it caused a `NoSuchBeanDefinitionException` at test runtime.

Resolution: The `@MockBean` annotation from Spring Boot Test was used to declare a Mockito mock of `StudentService` directly within the test class. This registers the mock as a Spring bean for the duration of the test, satisfying the Controller's dependency injection requirements.

11.2 Logback Configuration Conflicts

Placing `logback-spring.xml` alongside the default Spring Boot logging auto-configuration initially caused duplicate log output — messages were appearing twice in the console due to both the default and custom appenders being active simultaneously.

Resolution: The `additivity="false"` attribute was set on the custom logger element in `logback-spring.xml` for the application package. This prevented log events from propagating to the root logger's appenders, eliminating duplication.

11.3 Hibernate DDL and Existing Schema

During early development iterations, switching between `spring.jpa.hibernate.ddl-auto=create` and `update` caused the students table to be dropped and recreated on each restart, wiping test data that had been manually inserted via Postman for verification purposes.

Resolution: The DDL mode was set to `update` permanently once the schema was stable. The `update` mode adds new columns and tables but never drops existing structures, preserving data between restarts while still accommodating schema changes.

11.4 Mockito Argument Matching with any()

In the Controller test for `POST /students`, the Mockito stubbing initially used `when(studentService.saveStudent(student)).thenReturn(saved)`. However, because the JSON deserialisation creates a new object instance not equal to the one used in the stub, the mock did not match and returned null, causing the test to fail.

Resolution: The argument was replaced with `Mockito.any(Student.class)`, which matches any non-null `Student` argument regardless of object identity. This resolved the mismatch and allowed the stub to activate correctly during the `MockMvc` request.

12. Learning Outcomes

Completion of Task 3 delivered substantial hands-on learning across testing, logging, and Spring Boot development practices:

12.1 Unit Testing with JUnit 5 and Mockito

Gained practical experience designing and executing meaningful unit tests that isolate individual application components from their collaborators. Learned to use `@Mock`, `@InjectMocks`, `when().thenReturn()`, `verify()`, and assertion methods to validate method behaviour and interaction contracts without touching the real database or HTTP stack. Understood the difference between `@ExtendWith(MockitoExtension.class)` for pure Mockito tests and `@WebMvcTest` for Spring web layer tests.

12.2 Structured Logging with SLF4J and Logback

Developed an understanding of the SLF4J facade pattern and how it decouples application code from the underlying logging implementation. Learned to use the `@Slf4j` Lombok annotation, select appropriate log levels for different event types, and configure Logback via `logback-spring.xml` to produce formatted, persistent log files with rolling policies. Recognised the operational value of structured logs for debugging and monitoring.

12.3 Spring Boot Test Slices

Learned the distinction between full integration tests (`@SpringBootTest`) and lightweight test slices (`@WebMvcTest`, `@DataJpaTest`). Test slices load only the beans relevant to the layer under test, reducing startup time and isolating failures to specific layers — a critical pattern in professional Spring Boot test suites.

12.4 REST API Design and Postman Testing

Reinforced understanding of RESTful principles by implementing and testing GET and POST endpoints. Gained confidence using Postman to compose HTTP requests with JSON payloads, inspect response status codes, validate response body structure, and document API behaviour through saved collections.

12.5 Layered Architecture and Dependency Inversion

Deepened understanding of how Spring's dependency injection framework enables each layer to depend on abstractions (interfaces) rather than concrete implementations, making layers independently testable and replaceable. The combination of `@Service`, `@Repository`, and `@RestController` annotations with constructor injection enforces clean boundaries that are essential for enterprise-grade Spring Boot applications.

13. Conclusion

Task 3 of the InternSpark AICTE-listed Java Internship Program successfully delivered a complete Spring Boot REST API with MySQL integration, comprehensive unit testing using JUnit 5 and Mockito, and structured application logging using SLF4J and Logback.

The unit test suite validates both the Service and Controller layers in full isolation from their dependencies. The four test cases cover the two primary business operations — retrieving all students and persisting a new student — and verify both return values and interaction contracts with mocked collaborators. All four tests pass successfully in IntelliJ IDEA's JUnit runner.

The logging implementation instruments every significant code path with appropriately levelled SLF4J log statements, producing both console output and a rolling log file. The custom logback-spring.xml configuration provides a professional, production-aligned logging setup with timestamped, formatted entries.

The REST API — comprising GET /students and POST /students — was tested end-to-end using Postman, confirming correct HTTP status codes, accurate JSON response structures, and successful database persistence in the MySQL task3_db schema.

This task has meaningfully advanced the practical skill set required for professional Java backend development. The combination of unit testing, structured logging, and layered REST API design covered in this report represents a foundational competency in enterprise Spring Boot engineering, and provides a solid platform for the more complex tasks that follow in the internship program.

Submitted By

Swapnil Soni

B.Tech CSE — VIT Bhopal University
InternSpark AICTE-Listed Internship
Program

14. References

1. Spring Boot Documentation — <https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. Spring Data JPA Documentation — <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
3. JUnit 5 User Guide — <https://junit.org/junit5/docs/current/user-guide/>
4. Mockito Documentation — <https://site.mockito.org/>
5. SLF4J Manual — <https://www.slf4j.org/manual.html>
6. Logback Documentation — <https://logback.qos.ch/documentation.html>
7. Baeldung — Guide to Unit Testing with JUnit 5 and Mockito — <https://www.baeldung.com/mockito-junit-5-extension>
8. Baeldung — @WebMvcTest in Spring Boot — <https://www.baeldung.com/spring-boot-testing>
9. MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
10. Lombok Documentation — <https://projectlombok.org/features/>
11. InternSpark AICTE Internship Program Guidelines — <https://internspark.in/>

— End of Report —